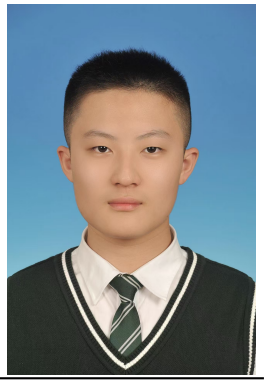
Computational Physics Hw_6
Project 2 Report

Student ID: 2024141230044

Name: 范奕



Compute π on a notebook computer with at least 10000 correct digits after the decimal point.

Pay attention to computational speed.

Under the later one-minute requirement, test how many digits can be computed reliably within one minute.

Contents

1 Source Code **3**

1.1 Program organization 3

1.2 Representative code fragments 3

2 Theoretical Analysis **4**

2.1 Why the Chudnovsky formula is suitable 4

2.2 Binary splitting recurrence 5

2.3 Parallel design choices 5

3 Numerical Results **6**

3.1 Test environment 6

3.2 Baseline tests 6

3.3 One-minute challenge 7

4 Discussion **8**

4.1 Performance assessment 8

4.2 Reliability boundary 8

4.3 Conclusion 8

1 Source Code

The implementation is contained in `main.c`, and the compiled executable is named `pi.exe`. The suffix `.exe` is only a filename convention here; the binary is actually a Linux ELF executable. The program combines four main ideas:

(1) the Chudnovsky series for fast convergence;

(2) binary splitting for efficient large-integer accumulation;

(3) GMP and MPFR for high-precision arithmetic;

(4) OpenMP task parallelism for multi-core acceleration.

1.1 Program organization

The source code can be divided into the following functional blocks:

(1) **Timing and constants.** The code stores the Chudnovsky constants and measures the binary splitting stage and the final floating-point evaluation stage separately.

(2) **Leaf kernel.** The function `bs_leaf_fast()` evaluates a short interval directly and reuses thread-local temporary variables to avoid repeated dynamic allocation inside the innermost loop.

(3) **Recursive binary splitting.** The functions `bs_serial()` and `bs_parallel()` compute the (P, Q, T) triple over an interval and merge two subintervals with the standard binary splitting recurrence.

(4) **Final evaluation of π.** After the integer stage is completed, the program converts the large integers to MPFR variables and evaluates

$$\pi = \frac{426880\sqrt{10005}\,Q}{T}.$$

1.2 Representative code fragments

The first key fragment shows how the target precision and the number of series terms are chosen:

```
long digits = 200000000;
if (argc >= 2) digits = atol(argv[1]);

long terms = digits / 14 + 10;
mpfr_prec_t prec = (mpfr_prec_t)(digits * 3.321928094887) + 256;
```

The second fragment is the binary splitting merge rule used in both the serial and parallel versions:

```
mpz_mul(P, P1, P2);
mpz_mul(Q, Q1, Q2);

mpz_mul(T, T1, Q2);
mpz_mul(tmp, P1, T2);
mpz_add(T, T, tmp);
```

The third fragment is the final MPFR evaluation:

```
mpfr_set_ui(sqrt10005, 10005, MPFR_RNDN);
mpfr_sqrt(sqrt10005, sqrt10005, MPFR_RNDN);
mpfr_mul_ui(factor, sqrt10005, 426880, MPFR_RNDN);
mpfr_set_z(qf, Q, MPFR_RNDN);
mpfr_set_z(tf, T, MPFR_RNDN);
mpfr_mul(pi, factor, qf, MPFR_RNDN);
mpfr_div(pi, pi, tf, MPFR_RNDN);
```

These fragments are sufficient to show the main algorithmic route without reproducing the full source file in the report.

2 Theoretical Analysis

2.1 Why the Chudnovsky formula is suitable

The Chudnovsky series is one of the standard choices for high-precision computation of π because it converges extremely fast. A convenient form is

$$\frac{1}{\pi} = \frac{12}{640320^{3/2}} \sum_{k=0}^{\infty} \frac{(-1)^k (6k)!\,(13591409 + 545140134k)}{(3k)!(k!)^3 640320^{3k}}.$$

Equivalently,

$$\pi = \frac{426880\sqrt{10005}}{\sum_{k=0}^{\infty} \frac{(-1)^k (6k)!\,(13591409 + 545140134k)}{(3k)!(k!)^3 640320^{3k}}}.$$

This series contributes about fourteen decimal digits per term, so the required number of terms grows only linearly with the requested number of digits. That is why the program uses

$$\texttt{terms} \approx \frac{\texttt{digits}}{14} + 10.$$

For a project that emphasizes both speed and scale, this is a very effective starting point.

2.2 Binary splitting recurrence

Directly evaluating each term and summing them one by one would generate very large intermediate numerators and denominators and would waste a great deal of time. To avoid that, the code uses binary splitting.

Define

$$p(k) = (6k-5)(2k-1)(6k-1), \qquad q(k) = k^3 \cdot \frac{640320^3}{24},$$

and let the linear factor be

$$a(k) = 13591409 + 545140134k.$$

For an interval $[l, r)$, the algorithm recursively computes a triple (P, Q, T). If the interval is split at m, the two child results (P_1, Q_1, T_1) and (P_2, Q_2, T_2) are merged by

$$P = P_1 P_2, \qquad Q = Q_1 Q_2, \qquad T = T_1 Q_2 + P_1 T_2.$$

This formulation turns a long sequential sum into a balanced recursion tree. Large multiplications and additions are grouped in a way that is much more suitable for both high-precision arithmetic libraries and parallel execution.

2.3 Parallel design choices

The current code contains several practical optimization ideas:

(1) A relatively large leaf size reduces recursion overhead.

(2) A task threshold prevents the program from creating too many tiny OpenMP tasks.

(3) Thread-local temporary `mpz_t` variables are allocated once and then reused inside the leaf kernel.

(4) The parallel function not only splits the left and right recursive branches, but also parallelizes several top-level large multiplications during the merge stage.

These details explain why the program is able to reach the 10^8-digit scale on a notebook CPU within a few tens of seconds.

3 Numerical Results

3.1 Test environment

All measurements below were obtained by directly running the existing executable in the current working environment. The main hardware and runtime information is summarized in Table 1.

表 1: Hardware and runtime environment.

Item	Value
CPU model	Intel(R) Core(TM) Ultra 9 185H
Logical CPUs visible to the program	22
Physical cores per socket	16
Maximum CPU frequency	5100 MHz
Parallel runtime	OpenMP
High-precision libraries	GMP and MPFR
Default runtime thread count	22

3.2 Baseline tests

I first ran the program at several smaller and medium scales. The correctness check in this report is based on two conditions: the program must not print `nan`, and the printed first 50 digits must match the standard decimal expansion of π.

表 2: Baseline performance measurements.

Digits	Threads	Binary splitting / s	Final eval / s	Total / s	Status
10^4	22	0.001392	0.000158	0.001550	correct
10^5	22	0.014236	0.003392	0.017629	correct
10^6	22	0.075113	0.050991	0.126103	correct
10^8	22	13.637795	10.295109	23.932904	correct

These results already show that the assignment requirement of at least 10000 digits is exceeded by a very large margin. The program remains fast even at 10^8 digits, where a full run takes only about 23.93 seconds on this machine.

3.3 One-minute challenge

The later requirement was to test how many digits could be computed within one minute. I therefore pushed the program to larger values and recorded the observed boundary behavior. The results are listed in Table 3.

表 3: Boundary measurements for the one-minute challenge.

Digits	Binary splitting / s	Final eval / s	Total / s	Output status
1.20×10^8	16.535251	12.228387	28.763638	correct
1.25×10^8	17.626152	13.838697	31.464849	correct
1.26×10^8	17.888330	14.165257	32.053586	correct
1.27×10^8	18.585380	14.494814	33.080194	correct
1.28×10^8	18.218021	5.494493	23.712513	nan
1.50×10^8	21.470960	5.927596	27.398556	nan
2.00×10^8	31.206240	8.631185	39.837425	nan
2.20×10^8	35.360358	9.544363	44.904720	nan

This table reveals an important distinction. If one only looks at elapsed time, even 2.2×10^8 digits finishes within one minute. However, the output becomes nan at and above 1.28×10^8 digits in the present implementation, so those runs cannot be counted as successful computations of π.

Therefore, the best reliable result obtained in this test campaign is

$$\boxed{127000000 \text{ digits in } 33.080194 \text{ seconds.}}$$

4 Discussion

4.1 Performance assessment

From the perspective of the original assignment, the program is already far beyond the minimum target. The requirement was at least 10000 digits, while the current implementation can reliably reach 127000000 digits on the tested notebook. The measured runtime at 10^8 digits, about 23.93 seconds, shows that the combination of the Chudnovsky series, binary splitting, and OpenMP task parallelism is highly effective.

The timing breakdown is also informative. At larger scales, the binary splitting stage remains the dominant cost, but the final MPFR evaluation is no longer negligible. At 10^8 digits, the final stage already takes about 10.30 seconds, which means that future optimization cannot focus only on the integer stage.

4.2 Reliability boundary

The most important finding of the present experiments is not only how fast the code runs, but where the current implementation stops producing valid output. In repeated tests, 1.27×10^8 digits still produced the correct leading digits of π, while 1.28×10^8 digits and above produced nan. That means the practical one-minute limit of this version is determined by correctness, not by raw runtime.

Based on the observed outputs alone, it would be unsafe to claim that the program truly computes 2×10^8 digits correctly, even though such runs terminate within one minute. The honest conclusion is that the current reliable upper bound on this machine is about 1.27×10^8 digits. Any larger claim would require further debugging and verification.

4.3 Conclusion

This project demonstrates that a notebook computer can compute an extremely large number of digits of π when an appropriate algorithm and implementation strategy are used. The present program uses the Chudnovsky formula, binary splitting, GMP/MPFR arithmetic, and OpenMP task parallelism to produce a reliable result of 127000000 digits in about 33.08 seconds.

The next improvement target should not be a larger nominal input alone. The more important next step is to identify why the output changes to nan above

1.27×10^8 digits and to raise the reliable boundary before claiming larger successful computations.

End of Report